

Reaching Clubuntu

Four separate obstacles stood between "I need to copy two files to clubuntu" and it actually happening. None were about the files.

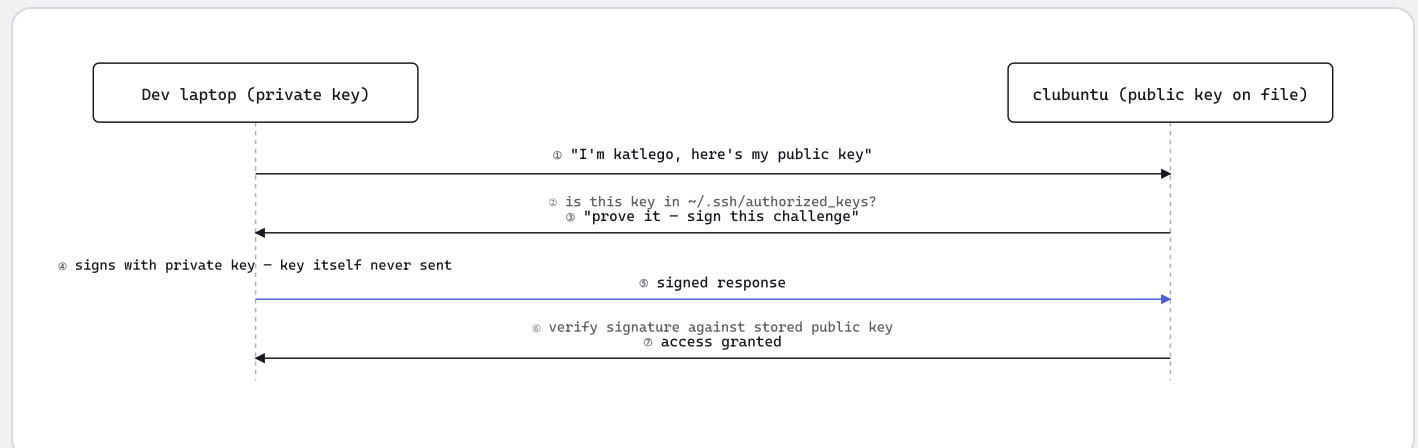
host: clubuntu.dala-cirius.ts.net 2026-08-16

01 The goal

Place `db/schema.sql` and `coder_commands.xlsx` at `/opt/docker/manpage` on clubuntu — the deploy compose file needs both there before its first run. Getting there required fixing SSH access, then fixing filesystem permissions, as two separate problems.

02 Obstacle 1 — no key authorized yet

SSH login by key works by proving possession of a private key without ever transmitting it. The public half has to be listed in the target account's `authorized_keys` first — it wasn't, so every connection attempt failed before anything else could happen.



Fix: appended the laptop's public key to `katlego`'s `authorized_keys` on clubuntu. This grants a new way to authenticate as an account that already existed — it creates nothing, permissions nothing on its own.

03 Obstacles 2 & 3 — no real terminal, then a key too long to type

- ▶ **Password auth failed silently.** No visible prompt, immediate rejection — not a wrong password, but this tool's command relay not providing SSH a real interactive terminal to prompt into at all.
- ▶ **The existing key was impractical to hand-type.** A 4096-bit RSA public key runs to ~3000 characters — the target session could be typed into but not pasted into (`Ctrl+V` and right-click both failed there).
- ▶ **Fix, two parts:** generated a dedicated ED25519 key just for this link — its public half is one short, typeable line — and found `Shift+Insert` worked as an alternate paste binding in that session anyway, making the whole block pasteable in one shot.

04 Obstacle 4 — logged in, still blocked

SSH decides *who* gets in. Once in, *what that user can touch* is a completely separate system — ordinary Unix ownership and permission bits, checked by the kernel on every file operation, with no awareness of how you authenticated.

BEFORE — owned by root

```
drwxr-xr-x root root
katlego is "other" here — read + enter only
X write denied — mkdir, scp both fail
```

AFTER — `chown -R katlego:katlego`

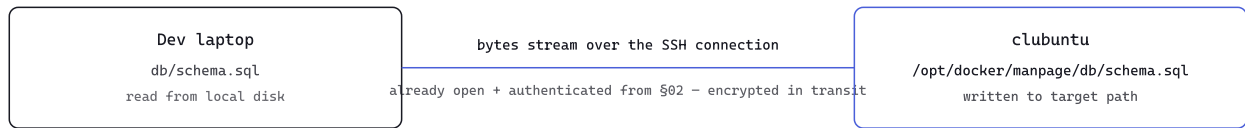
```
drwxr-xr-x katlego katlego
katlego is now the owner
✓ full read + write
```

same login, same key — only the filesystem grant changed

`sudo chown -R katlego:katlego /opt/docker/manpage` — needed its own interactive password prompt (a third, separate check: is this user in the `sudo` group, and do they know their own password), same real-terminal requirement the SSH password attempt lacked here.

05 The copy itself — scp is SSH, not a separate protocol

`scp` isn't a different tool bolted onto SSH — it's SSH's own file-transfer mode, reusing the exact same encrypted, authenticated connection from §02. No second login, no second key check: once the connection from §02 is open, the same channel just carries file bytes instead of (or alongside) a shell session.



afterward: sha256sum on both ends, compared – identical hashes confirm
every byte arrived exactly as sent, independent of trusting the transfer blindly

Two files, same mechanism: `db/schema.sql` to `/opt/docker/manpage/db/`, then
`coder_commands.xlsx` to `/opt/docker/manpage/`. The checksum step isn't scp verifying
itself — it's an independent check run separately on each side afterward.

06 Verified

```
$ sha256sum db/schema.sql coder_commands.xlsx (local)
1ad37a03 ... fea2271 db/schema.sql
9c341aa7 ... d90f15e coder_commands.xlsx

$ ssh clubuntu ... sha256sum ... (remote)
1ad37a03 ... fea2271 /opt/docker/manpage/db/schema.sql
9c341aa7 ... d90f15e /opt/docker/manpage/coder_commands.xlsx

identical, both files
```

07 Status of /opt/docker/manpage

ITEM	STATUS
<code>db/schema.sql</code>	✓ present, verified
<code>coder_commands.xlsx</code>	✓ present, verified
Directory ownership	✓ katlego:katlego
<code>.env</code>	still to create
<code>docker-compose.deploy.yml</code>	not yet — first deploy run copies it in automatically

